

Projeto Básico - Prova Piloto

Descrição: Este projeto básico é uma abordagem técnica bastante sólida e direta. A proposta de avaliação menciona explicitamente que "detalhes de UI/UX não fazem parte dos critérios da prova", o que tecnicamente viabiliza a criação de uma Interface de Linha de Comando (CLI) executada no terminal.

Ainda assim, como a prova exige funcionalidades visuais específicas como exibir listas de produtos, realizar pesquisas e exibir alertas de estoque mínimo, sua aplicação no terminal precisará organizar as saídas de texto de forma bem limpa para que o avaliador perceba que os requisitos da "Interface" foram cumpridos.

Para dar sentido aos dados, adotei o contexto de **fabricante de ferramentas e equipamentos manuais**.

Adicionei também o campo *estoque_minimo* na tabela de produtos, pois exige que o sistema dispare um alerta caso a saída de um item deixe o nível abaixo do mínimo. Abaixo, apresento a estrutura de projeto adaptada para o terminal.

Estrutura de Diretórios e Arquivos (CLI / Terminal)

nome_sobrenome/

├── ANEXO III - documentacao.docx

├── saep_db.sql

└── sistema/

 ├── README.md

 ├── requirements.txt

 ├── main.py

 ├── database.py

 ├── modulo_auth.py

 ├── modulo_produto.py

 └── modulo_estoque.py

Detalhamento e Mapeamento das Entregas

1. Diretório Raiz

O padrão de entrega exigido pelo manual se mantém intacto:

- **ANEXO III - documentacao.docx:** Arquivo contendo as entregas teóricas.
- **saep_db.sql:** Script do banco de dados.

2. Diretório do Sistema (sistema/)

- **README.md:** Instruções de como rodar o sistema no terminal.
- **requirements.txt:** Ficará vazio ou apenas com pacotes de formatação de terminal (como o pacote `tabulate`, se você quiser exibir as tabelas do banco de dados de forma mais elegante no console).
- **database.py:** Concentra a comunicação com o SQLite3. Terá funções que recebem parâmetros e retornam listas ou tuplas, sem se preocupar com print ou input.

3. Módulos de Interface via Terminal

- **main.py:** Este será o arquivo executado pela IDE. Ele conterá o laço `while True` principal. O menu deve exibir as opções numéricas para o usuário e apresentar o nome do usuário logado. Ele direcionará o fluxo para os outros módulos dependendo do `input()` digitado, além de possuir a opção de "Logout" para quebrar o fluxo e retornar à tela de login.

- **modulo_auth.py:** Conterá uma função com seu próprio fluxo que solicita via input() o usuário e senha. Ele validará os dados com o database.py. Se falhar, exibirá um print() com o motivo da falha. Se tiver sucesso, retornará os dados do usuário para o main.py iniciar o menu principal.
- **modulo_produto.py:** Terá funções chamadas pelo menu principal.
 - Uma função para exibir a lista de produtos (um SELECT impresso linha a linha no terminal).
 - Uma função de pesquisa usando input() para filtrar a lista.
 - Uma função de cadastro com um pequeno loop de validação, utilizando blocos try/except para impedir que o usuário digite texto em campos onde o sistema espera números.
- **modulo_estoque.py:** Apresentará a lista de produtos em ordem alfabética. Ao solicitar o ID do produto para movimentação, o sistema pedirá um input() para definir se é Entrada ou Saída, seguido do input() da data. O sistema processará a lógica e fará um print() de destaque (talvez usando caracteres especiais como !!! ALERTA !!!) caso o estoque fique abaixo do mínimo.